

TrackMind AI — Full Prototype Technical Specification

AI-Powered Automatic Block Planning & Corridor Management System for Indian Railways

Smart India Hackathon • Problem Statement 26027 | System Architecture & Comprehensive Technical Documentation

1. Executive Summary & Problem Statement

Indian Railways operates one of the densest rail networks globally. Maintenance disconnections ('blocks') are independently requested by three departments: **Engineering (Track/TMS)**, **Signal & Telecom (S&T;/SMMS)**, and **Traction Distribution (TRD/TDMS)**. Operating in departmental silos via the Block Demand Management System (BDMS) creates critical operational inefficiencies:

- **Departmental Conflicts:** Multiple departments request overlapping blocks on the same section at different times, causing redundant line shutdowns.
- **Timetable Disruption:** Uncoordinated blocks collide with scheduled passenger trains and high-priority freight rakes, escalating delay hours across railway divisions.
- **Asset Underutilization:** Heavy track machinery (tie tampers, ballast cleaners) sits idle awaiting manual safety clearance.
- **Lack of Real-Time Intelligence:** Maintenance decisions are reactive rather than predictive and risk-optimized.

The TrackMind AI Solution: Our prototype introduces a fully automated, agentic AI block planning engine. It ingests multi-department defect records, corridor availability (COA), live timetables, and goods traffic forecasts. Using machine learning priority scoring, spatial anomaly detection, and a Google OR-Tools CP-SAT mathematical solver, TrackMind AI schedules maintenance blocks with **zero departmental collisions** and enforced timetable constraints while maximizing asset availability.

2. Complete Technology Stack & Architecture ('What We Used')

The system is engineered using open-source tools selected for performance, reliability, and ease of deployment:

Component Layer	Technologies & Libraries Used	Key Purpose & Functionality
Frontend UI Framework	Streamlit, HTML5, Custom CSS3, Plotly Express	Multi-role interactive dashboard (Admin, Engineering, S&T;, TRD) with live Plotly Gantt timeline graphs.
Backend & Relational DB	Python 3.13, SQLite3 database	Lightweight relational database storing all defects, timetables, schedules, notifications, and audit logs.
Optimization Engine	Google OR-Tools (CP-SAT Solver)	Constraint programming solver enforcing hard zero-collision constraints and maximizing priority scores.
Machine Learning Layer	Scikit-Learn (RandomForest, IsolationForest), Pandas, NumPy	Predicts defect failure risk probability and identifies spatial defect clustering across 40 section IDs.
Agentic AI Architecture	Plain-Python OOP Framework (32 Agents)	Decoupled autonomous agents handling SLA compliance, cost optimization, crew rosters, and advisories.
Conversational & Voice AI	Groq Cloud API (llama-3.3-70b), Web Speech API	Multilingual voice assistant (English, Hindi, Telugu) and direct text-to-SQL time-window querying.

Document & PDF Engine	ReportLab, fpdf2	Generates official downloadable block summary reports, audit trails, and technical documentation.
Synthetic Data Generator	Faker, Python Random, DateTime	Generates 2,000+ realistic records per department across 40 sections in 5 railway divisions.

3. Data Architecture & Database Schemas (`scripts/generate\_data.py` & `setup\_database.py`)

The system simulates realistic Indian Railways operations across **5 Divisions** (Secunderabad, Vijayawada, Guntakal, Guntur, Hyderabad) containing **40 Section IDs**. Data is persisted in an indexed SQLite database (`railway.db`).

Database Table	Key Columns / Attributes	Description & Domain Purpose
defects	defect_id, department, section_id, defect_type, severity, reported_date, due_date, overdue_days, estimated_duration_hours, trains_affected_per_day, status, priority_score, risk_score	Registry of all disconnections requested by Engineering (TMS), S&T; (SMMS), and TRD (TDMS). Stores ML score outputs.
corridor_slots	slot_id, section_id, date, start_time, end_time, duration_hours, slot_type, is_available	Corridor maintenance windows granted by Control Office (Day block, Night block, Maintenance window).
train_timetable	train_no, train_name, train_type, section_id, arrival_time, departure_time, frequency	Official passenger train movement timeline used as a hard constraint to prevent block collisions.
goods_forecast	forecast_id, section_id, date, expected_rakes, priority_level	Freight traffic demand forecasts (FOIS) protecting revenue-earning goods corridors from block overbooking.
schedule	schedule_id, defect_id, slot_id, section_id, department, planned_start, planned_end, horizon, status, decided_by	Final optimized block allocations output by CP-SAT solver and confirmed by Section Controllers.
audit_log & notifications	actor, action, details, timestamp, recipient_role, message, category, audience	Full audit trail recording agent decisions, controller overrides, and multi-department notification alerts.

4. Machine Learning Intelligence Layer (`scripts/scoring\_models.py`)

Prior to optimization, every defect is evaluated by a dual ML/rule-based intelligence pipeline:

A. Multi-Factor Priority Scoring Model

Defect urgency is calculated using a multi-factor formula that balances safety severity, overdue delay, and operational impact:

Priority Score = Severity Weight + Overdue Component + Impact Component

Where:

- **Severity Weight:** Critical = 40, High = 25, Medium = 12, Low = 5.
- **Overdue Component:** min(Overdue Days, 90) / 90 * 30 (scales 0–30 based on SLA overdue status).
- **Impact Component:** min(Trains Affected per day, 50) / 50 * 30 (scales 0–30 based on corridor traffic density).

B. Predictive Failure Risk Model (Random Forest)

A RandomForestClassifier (20 estimators, max depth 6) trains on historical defect features [*severity_component*, *overdue_component*, *impact_component*, *estimated_duration_hours*] to estimate the probability that a defect will fail prior to maintenance. The final ranking blends priority and predicted risk: **Final Score = 0.6 × Priority + 0.4 × Risk Probability**.

C. Spatial Anomaly Detection Engine (Isolation Forest)

An IsolationForest model (contamination = 0.15) evaluates open defect counts and average overdue days across all 40 section IDs. Sections exhibiting abnormal defect clustering are flagged as infrastructure anomalies, notifying senior divisional engineers.

5. Mathematical Optimization Engine (`scripts/optimizer.py`)

The scheduling engine utilizes **Google OR-Tools CP-SAT** (Constraint Programming - Satisfiability) to solve the combinatorial block allocation problem across weekly (7-day) and monthly (30-day) horizons.

Mathematical Formulation & Hard Constraints:

- 1. **Slot Capacity Constraint:** At most one defect task can be assigned to any corridor slot (zero departmental clashes).
- 2. **Task Uniqueness Constraint:** Each defect task is assigned to at most one slot per optimization cycle.
- 3. **Spatial & Duration Fit:** Task *i* can only pair with slot *j* if defect.section_id == slot.section_id and defect.estimated_duration <= slot.duration.
- 4. **Train Timetable Exclusion (Hard Constraint):** filter_slots_against_timetable() automatically filters out any slot whose window overlaps a scheduled passenger train departure on that section.
- 5. **Controller Lock Enforcement:** filter_slots_against_locked_schedules() excludes slots overlapping existing locked or controller emergency override schedules.

Objective Function: Maximize total scaled priority score of all scheduled maintenance tasks:

Maximize $\sum_{(i,j)} (\text{Priority Score}_i \times 100) \cdot x_{i,j}$

6. Autonomous Agentic AI Layer (`scripts/agents.py`)

TrackMind AI incorporates a modular agent framework comprising **32 specialized autonomous agents** implemented as clean Python classes. Agents interact by updating state directly in SQLite database tables.

Agent Category	Key Autonomous Agents	Responsibilities & Operational Logic
Department Agents	DepartmentAgent (Engg, S&T;, TRD)	Filters, ranks, and submits department-specific block requests based on defect urgency.
Core Coordination	TrafficAgent, CoordinatorAgent, ReplanningAgent	Protects freight/timetable corridors, executes CP-SAT optimizer, and triggers dynamic re-planning when emergency defects occur.
Governance & SLA	DeadlineAlertAgent, ComplianceAgent, CostOptimizationAgent	Monitors overdue SLA days, tracks compliance percentages, and estimates economic impact of train delays.
Field & Roster Ops	CrewAvailabilityAgent, TrackMachinePackerAgent, LocopilotSpeedAgent	Verifies crew duty hours (HOER rules), schedules tie-tamping machines, and issues TSR speed restriction advisories.

Network Capacity	FreightInsertionAgent, SingleLineWorkingAgent, FOISDemurrageAgent, DeBunchingMeteringAgent	Inserts ad-hoc freight rakes during mini-gaps, manages single-line working on double tracks, and minimizes demurrage penalties.
Safety & Risk	SafetyClearanceAgent, OperationalRiskAgent, TSRLifecycleAgent	Enforces electrical/track safety clearances, calculates section risk indices, and manages Temporary Speed Restrictions.

3-Tier Auto-Approval Risk Hierarchy:

- **Tier 1 (Low Risk):** Night blocks, duration ≤ 2h, non-trunk sections. *Auto-approved and committed immediately.*
- **Tier 2 (Medium Risk):** Day blocks, duration 2–4h. *Provisional schedule generated; flagged for Section Controller confirmation.*
- **Tier 3 (High Risk):** Major trunk corridors, duration > 4h. *Requires manual emergency override by Senior Divisional Operations Manager (Sr. DOM).*

7. User Interface & Multilingual Voice AI Assistant (`app/main.py` & `chatbot.py`)

The frontend is built using Streamlit and provides a clean, role-based dashboard for railway controllers:

- **Role-Based Dashboards:** Supports Admin (Central Controller, 10 full tabs), Engineering (Track), S&T;, and TRD views with bcrypt password authentication.
- **Interactive Timeline & Plotly Analytics:** Visual Block Windows Timeline renders all maintenance tasks in zoomable Plotly Gantt charts with permanent axis labels.
- **Multilingual Voice AI Assistant:** Integrated right-side AI panel powered by Groq Cloud API (`llama-3.3-70b-versatile`) with automated model fallback. Utilizes the browser Web Speech API for voice recognition and text-to-speech in **English** (`en-IN`), **Hindi** (`hi-IN`), and **Telugu** (`te-IN`).
- **Direct SQL Time-Window Querying:** Enables natural language queries such as *'What is scheduled between 2pm and 4pm on Vijayawada section?'* by parsing time windows directly into SQLite queries.

8. PDF Reporting Pipeline (`app/reports.py`)

TrackMind AI includes an automated PDF generation engine (`app/reports.py`) powered by fpdf2. Controllers can generate official, downloadable weekly and monthly block analysis reports detailing completion metrics, departmental summaries, and scheduled block details for executive reporting and audit compliance.

9. Step-by-Step Build Process ('How We Made It')

The development of TrackMind AI followed a structured 8-step engineering workflow:

- **Step 1: Domain Analysis & Problem Definition:** Analyzed SIH Problem Statement 26027, mapping railway BDMS, TMS, SMMS, TDMS, and COA workflows to identify key pain points in disconnection planning.
- **Step 2: Synthetic Data Generation Pipeline:** Wrote `scripts/generate\_data.py` using `faker` to synthesize 2,000+ records per dataset across 40 sections in 5 divisions, ensuring realistic defect distributions and timetable parameters.
- **Step 3: Database & Schema Initialization:** Developed `scripts/setup\_database.py` to construct the indexed SQLite schema (`railway.db`), auto-load CSV datasets, and seed hashed default user credentials.
- **Step 4: Machine Learning Intelligence Layer:** Implemented `scripts/scoring\_models.py` featuring the priority scoring algorithm, RandomForest failure risk estimator, and IsolationForest anomaly detector.

- **Step 5: Mathematical CP-SAT Optimizer Integration:** Built ``scripts/optimizer.py`` using Google OR-Tools CP-SAT solver, encoding slot uniqueness, duration fit, timetable clash prevention, and priority maximization.
- **Step 6: Agentic Architecture & Orchestration Workflow:** Created ``scripts/agents.py`` defining 32 specialized autonomous Python agents and the ``CoordinatorAgent.run_cycle()`` master execution flow.
- **Step 7: Streamlit Dashboard & Voice AI Integration:** Developed ``app/main.py`` and ``app/chatbot.py`` to deliver a multi-role UI, interactive Plotly Gantt timelines, Groq LLM integration, and Web Speech API voice synthesis.
- **Step 8: Automated PDF Reporting & End-to-End Testing:** Built ``app/reports.py`` for automated PDF report generation and created an end-to-end CLI test suite verifying scoring, optimization, and agent cycles.

10. End-to-End Operational Lifecycle ('How It Happens Step-by-Step')

When deployed in a Railway Central Control Office, a block request moves through an automated 7-step lifecycle:

Step #	Phase / Stage	Operational Execution Details
1	Defect Ingestion	Engineering, S&T,, and TRD log defect disconnections or voice-input tasks via BDMS/Streamlit dashboard into defects table.
2	ML Scoring & Anomaly Flagging	<code>compute_priority_scores()</code> computes priority scores and failure risks; <code>detect_anomalies()</code> flags defective track clusters.
3	CP-SAT Optimization	<code>run_optimizer()</code> filters slots against train timetables and locked schedules, then solves CP-SAT model to allocate blocks with zero clashes.
4	Agent Conflict & SLA Check	32 agents execute in cycle: <code>TrafficAgent</code> protects freight rakes, <code>ComplianceAgent</code> tracks SLAs, <code>CrewAgent</code> verifies rosters, <code>SafetyAgent</code> validates clearances.
5	Controller Review & Approval	Tier 1 auto-approves. Tier 2/3 flag provisional plans on Central Controller Dashboard for one-click confirmation or emergency override.
6	Automated Advisory Dispatch	<code>PassengerAdvisoryAgent</code> posts station bulletins; <code>LocopilotSpeedAgent</code> issues TSR advisories; <code>CrewAgent</code> dispatches machine operators.
7	Real-Time Tracking & Audit	Field engineers mark completion; <code>FeedbackLoopAgent</code> updates defect status; Audit log records complete execution history.